

Software Requirements Specification (SRS)

ArtAuction — Microservices Platform

Project: Full-Stack Microservices Application

Backend: Spring Boot 3 · Spring Cloud · PostgreSQL · Docker Compose

Frontend: React 18 · Vite · Axios · WebSockets

Table of Contents

1. [Introduction](#)
 - 1.1 Purpose
 - 1.2 Project Scope
2. [Overall Description](#)
 - 2.1 Product Perspective
 - 2.2 Product Features
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 User Documentation
 - 2.7 Assumptions and Dependencies
3. [System Features](#)
 - 3.1 User Management & Authentication
 - 3.2 Artwork Catalogue Management
 - 3.3 Auction Management
 - 3.4 Bidding System
 - 3.5 Notification System
 - 3.6 Real-Time Features
 - 3.7 Role-Based Access Control (RBAC)
 - 3.8 Watchlist Management
4. [External Interface Requirements](#)

- 4.1 User Interfaces
- 4.2 Hardware Interfaces
- 4.3 Software Interfaces
- 4.4 Communications Interfaces

5. Non-Functional Requirements

- 5.1 Performance Requirements
- 5.2 Security Requirements
- 5.3 Availability & Reliability
- 5.4 Maintainability
- 5.5 Scalability

6. UML Diagrams

- 6.1 Use Case Diagram
- 6.2 Activity Diagrams
- 6.3 Sequence Diagrams
- 6.4 Class Diagram
- 6.5 Entity-Relationship Diagram (ERD)

7. Implementation Details

- 7.1 API Specifications
- 7.2 Object Constraint Language (OCL)
- 7.3 Aspect-Oriented Programming (AOP)
- 7.4 Docker Configuration
- 7.5 Clean Code Practices
- 7.6 Design Patterns
- 7.7 Microservices & Cloud Architecture

8. Test Results

- 8.1 postman
- 8.2 Frontend
- 8.3 Databases

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for the **ArtAuction Microservices Platform** — a full-stack web application that enables artists to auction artworks, buyers to place bids, and administrators to manage the platform. This document serves as the authoritative reference for development, testing, and project evaluation.

The system implements a modern microservices architecture with a React frontend, Spring Boot backend services, PostgreSQL databases, and Docker containerization.

1.2 Project Scope

The ArtAuction platform provides: - **User registration and authentication** with role-based access - **Artwork catalogue management** for artists - **Auction creation, scheduling, and management** - **Real-time bidding system** with bid history - **Notification system** for auction events - **Watchlist functionality** for buyers - **Admin moderation tools** for user and content management - **Responsive web interface** built with React and Vite

Out of Scope: Payment processing, shipping logistics, third-party authentication (OAuth), mobile native applications.

2. Overall Description

2.1 Product Perspective

The ArtAuction platform is a self-contained web application consisting of: - **Frontend:** React 18 SPA (Single Page Application) served via Vite - **Backend:** Five Spring Boot microservices (Gateway, User, Auction, Notification, Watchlist) - **Infrastructure:** Eureka service discovery, PostgreSQL databases, Docker Compose orchestration

2.2 Product Features

Feature		
ID	Feature Name	Description
1	User Registration	Register with role selection (ADMIN, EMPLOYEE, ARTIST, BUYER)
2	User Authentication	JWT-based login/logout with 24h token expiry
3	Artist Approval	ADMIN/EMPLOYEE approves pending artists before login
4	Artwork Upload	ARTIST uploads artwork with metadata and starting price
5	Auction Creation	ARTIST creates auction linked to artwork with schedule
6	Auction Scheduling	Automatic activation/closing via scheduler
7	Bid Placement	BUYER places bids with amount validation
8	Bid History	Public and private bid history viewing
9	Real-Time Notifications	WebSocket-based notification delivery
10	Watchlist Management	BUYER saves auctions to personal watchlist via dedicated microservice
11	User Management	ADMIN views/deletes users, toggles status
12	Content Moderation	EMPLOYEE cancels auctions, deletes artworks
13	Search & Filter	Browse auctions with filters and search
14	Profile Management	Users edit profiles and view statistics

2.3 User Classes and Characteristics

User Class	Description
ADMIN	System administrator with full control

User Class	Description
EMPLOYEE	Gallery staff for moderation
ARTIST	Content creators uploading artworks
BUYER	End users placing bids
GUEST	Unauthenticated visitors browsing

2.4 Operating Environment

Server Environment: - Docker Desktop >= 4.x with Compose v2 - 6 GB RAM recommended - Java 17+ (for Spring Boot) - Node.js 18+ (for React build)

Client Environment: - Modern web browsers (Chrome, Firefox, Safari, Edge) - Minimum resolution: 320px (mobile responsive) - JavaScript enabled

Network: - Ports required: 5432, 8080, 8081, 8082, 8083, 8761 - Local development: `http://localhost:8080`

2.5 Design and Implementation Constraints

Constraint	Description
Microservices	Must use separate deployable services for User, Auction, and Notification domains
JWT Security	All authenticated requests must include valid Bearer tokens
Database Isolation	Each service must have its own PostgreSQL database
Docker Deployment	Entire system must run via docker <code>compose up</code>
API Gateway	All client requests must route through gateway at port 8080
Frontend Tech Stack	Must use React 18, Vite, react-router-dom, and axios

Constraint	Description
AOP Logging	All services must implement cross-cutting logging via Spring AOP

2.6 User Documentation

- **API Documentation:** Swagger UI available at <http://localhost:8081/swagger-ui.html>, 8082, 8083
- **Setup Guide:** README with Docker Compose instructions
- **Frontend README:** Component structure and proxy configuration

2.7 Assumptions and Dependencies

Assumptions: - Users have reliable internet connectivity - Docker Desktop is properly configured - Ports specified are available on the host machine - Artists provide accurate artwork information

Dependencies: - Spring Boot 3.2+, Spring Cloud 2023, Spring Security - React 18, Vite 5, react-router-dom 6, axios - PostgreSQL 15, Docker Compose - Eureka Server for service discovery

3. System Features

3.1 User Management & Authentication

3.1.1 Description

The system shall provide comprehensive user management with role-based registration, JWT authentication, and artist approval workflow.

3.1.2 Functional Requirements

- 1:** The system shall allow user registration with fields: fullName, email, password, role, and optional bio (for ARTIST).
- 2:** The system shall validate email uniqueness across all users.
- 3:** The system shall hash passwords using BCrypt before storage.
- 4:** The system shall support four roles: ADMIN, EMPLOYEE, ARTIST, BUYER.
- 5:** The system shall require ADMIN/EMPLOYEE approval for ARTIST accounts before first login.

- 6:** The system shall generate JWT tokens upon successful login with 24-hour expiry.
- 7:** The system shall validate JWT tokens at the API Gateway for every incoming request.
- 8:** The system shall allow public access (no token) to GET /api/artworks and GET /api/auctions.
- 9:** The system shall allow ADMIN to list all users and delete any user.
- 10:** The system shall allow EMPLOYEE to list users and approve pending artists.

3.1.3 Use Case: User Registration

Use Case ID: 1

Use Case Name: Register User

Actor: Guest

Preconditions: User is not authenticated

Postconditions: User account created (pending approval if ARTIST)

Main Flow: 1. Guest navigates to registration page 2. System displays registration form with role selection 3. Guest enters fullName, email, password, selects role 4. If role is ARTIST, guest enters bio 5. Guest submits form 6. System validates email uniqueness 7. System hashes password and stores user 8. If ARTIST, system sets status to PENDING 9. System returns success message

Alternative Flows: - 6a. Email exists: System returns 409 Conflict - 5a. Invalid data: System returns 400 Bad Request with validation errors

3.2 Artwork Catalogue Management

3.2.1 Description

Artists shall upload and manage artworks with detailed metadata. Artworks serve as the foundation for auctions.

3.2.2 Functional Requirements

- 1:** The system shall allow ARTIST to upload artwork with: title, description, medium, dimensions, yearCreated, startingPrice.
- 2:** The system shall validate that startingPrice > 0.

- 3:** The system shall associate artwork with the uploading ARTIST.
 - 4:** The system shall allow public viewing of all artworks via GET /api/artworks.
 - 5:** The system shall allow EMPLOYEE/ADMIN to delete any artwork.
 - 6:** The system shall prevent deletion of artworks linked to active auctions.
-

3.3 Auction Management

3.3.1 Description

The system shall support auction lifecycle management from creation through scheduling to automatic closure.

3.3.2 Functional Requirements

- 1:** The system shall allow ARTIST to create auction linked to their artwork.
 - 2:** The system shall require auction fields: artworkId, startTime, endTime, reservePrice.
 - 3:** The system shall validate that $startTime < endTime$.
 - 4:** The system shall set auction status to SCHEDULED upon creation.
 - 5:** The system shall automatically activate SCHEDULED auctions when $startTime \leq currentTime$ (via 30s scheduler).
 - 6:** The system shall automatically close ACTIVE auctions when $endTime \leq currentTime$ and determine winner.
 - 7:** The system shall allow ARTIST to cancel their own auctions (if not closed).
 - 8:** The system shall allow EMPLOYEE/ADMIN to cancel any auction.
 - 9:** The system shall provide public endpoint GET /api/auctions/active.
 - 10:** The system shall prevent bids on non-ACTIVE auctions.
-

3.4 Bidding System

3.4.1 Description

Buyers shall place bids on active auctions with automatic validation of bid amounts and real-time history tracking.

3.4.2 Functional Requirements

- 1:** The system shall allow BUYER to place bid on active auction.

- 2:** The system shall validate bid amount > current highest bid (or starting price if first bid).
 - 3:** The system shall validate bid amount >= reservePrice (if first bid).
 - 4:** The system shall record bid with timestamp, amount, buyerId, and auctionId.
 - 5:** The system shall provide public bid history per auction.
 - 6:** The system shall allow users to view their own bid history.
 - 7:** The system shall update auction currentPrice upon valid bid placement.
-

3.5 Notification System

3.5.1 Description

The system shall deliver real-time and persistent notifications for auction events (new bid, auction won, auction closed, etc.).

3.5.2 Functional Requirements

- 1:** The system shall generate notifications for: new bid placed, auction won, auction closed, outbid, artist approved.
 - 2:** The system shall deliver notifications via WebSocket for real-time updates.
 - 3:** The system shall persist notifications in database.
 - 4:** The system shall allow users to view their notifications via GET `/api/notifications/my`.
 - 5:** The system shall allow ADMIN to view all notifications.
 - 6:** The system shall allow users to mark notifications as read.
 - 7:** The system shall display unread notification count in UI.
-

3.6 Real-Time Features

3.6.1 Description

The frontend shall establish WebSocket connections for real-time bid updates and notification delivery.

3.6.2 Functional Requirements

- 1:** The system shall establish WebSocket connection upon user login.
- 2:** The system shall push new bid events to all connected clients viewing the auction.

- 3:** The system shall push notification events to targeted users.
 - 4:** The system shall handle WebSocket reconnection on connection loss.
 - 5:** The system shall fallback to polling if WebSocket unavailable.
-

3.7 Role-Based Access Control (RBAC)

3.7.1 Description

The system shall enforce granular permissions based on user roles at both gateway and controller levels.

3.8 Watchlist Management (Frontend Feature)

3.8.1 Description

The frontend provides watchlist UI for buyers to save and track auctions. The actual watchlist persistence and business logic are handled by the dedicated Watchlist microservice.

3.8.2 Functional Requirements

- 1:** The frontend shall allow BUYER to add auction to watchlist via `/api/watchlist` endpoint.
 - 2:** The frontend shall allow BUYER to remove auction from watchlist.
 - 3:** The frontend shall display watchlist items with auction status and current price fetched from Watchlist Service.
 - 4:** The frontend shall show unread notification count when watched auction receives new bid.
-

3.9 Watchlist Service (Microservice)

3.9.1 Description

The Watchlist Service is a dedicated microservice (port :8084) responsible for managing buyer watchlists independently from the Auction Service. It maintains its own PostgreSQL database (`watchlist_service_db`) and communicates with other services via REST APIs and Feign clients.

3.9.2 Functional Requirements

- 1:** The Watchlist Service shall provide REST API for adding auctions to a buyer's watchlist.
- 2:** The Watchlist Service shall provide REST API for removing auctions from a buyer's watchlist.
- 3:** The Watchlist Service shall provide REST API for retrieving a buyer's complete watchlist with auction details.
- 4:** The Watchlist Service shall validate that the auction exists before adding to watchlist (via Feign call to Auction Service).
- 5:** The Watchlist Service shall prevent duplicate entries for the same buyer-auction pair.
- 6:** The Watchlist Service shall notify the Notification Service when a watched auction receives a new bid.
- 7:** The Watchlist Service shall maintain its own isolated database (watchlist_service_db).
- 8:** The Watchlist Service shall register itself with Eureka Server on startup.
- 9:** The Watchlist Service shall implement AOP logging consistent with other business services.

3.9.3 API Endpoints

Method	Endpoint	Auth	Role	Description
POST	/api/watchlist	Bearer	BUYER	Add auction to watchlist
DELETE	/api/watchlist/{auctionId}	Bearer	BUYER	Remove from watchlist
GET	/api/watchlist/my	Bearer	BUYER	Get my watchlist
GET	/api/watchlist/{auctionId}/watchers	Bearer	ADMIN/ EMPLO YEE	Get watchers count

3.9.4 Inter-Service Communication

From	To	Protocol	Purpose
Watchlist Service	Auction Service	OpenFeign	Validate auction exists
Watchlist Service	Notification Service	OpenFeign	Notify watchers of new bids
API Gateway	Watchlist Service	HTTP (lb://)	Route client requests
Watchlist Service	Eureka Server	HTTP	Service registration

4. External Interface Requirements

4.1 User Interfaces

Frontend Structure:

```
artauktion-frontend/
├── index.html           ← Entry HTML
├── vite.config.js       ← Vite config with API proxy
├── package.json         ← Dependencies: react-router-dom, axios
├── nginx.conf           ← Production nginx config
├── src/
│   ├── main.jsx         ← App entry point
│   ├── App.jsx          ← Router setup
│   ├── context/
│   │   ├── AuthContext.js ← Login/register/logout state
│   │   └── NotificationContext.js ← Real-time notifications
│   ├── services/
│   │   └── api.js        ← Axios API layer
│   ├── hooks/
│   │   ├── useAuctions.js ← Fetch auction lists with filters
│   │   └── useAuction.js  ← Fetch single auction
│   ├── utils/
│   │   └── formatters.js ← Price, date, time formatting
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Navbar.jsx ← Responsive nav with auth state
│   │   │   ├── Footer.jsx
│   │   │   └── *.css
│   │   ├── auction/
│   │   │   ├── AuctionCard.jsx ← Artwork card with bid info
│   │   │   ├── BidHistory.jsx  ← Bid list with rankings
│   │   │   ├── BidForm.jsx     ← Place bid form
│   │   │   └── *.css
│   └── auth/
```

```

└── ProtectedRoute.jsx ← Route guard
└── pages/
    ├── Home.jsx          ← Hero + featured auctions
    ├── Auctions.jsx       ← Browse with filters/search
    ├── AuctionDetail.jsx  ← Artwork page + bidding
    ├── Login.jsx / Register.jsx
    ├── Profile.jsx        ← Edit profile + stats
    ├── Watchlist.jsx      ← Saved auctions
    ├── Notifications.jsx  ← Real-time alerts
    ├── CreateAuction.jsx  ← Sell artwork form
    ├── NotFound.jsx
    └── *.css

```

Page Descriptions:

Page	Route	Description	Access
Home	/	Hero section, featured active auctions	Public
Auctions	/auctions	Browse all auctions with filters	Public
Auction Detail	/auctions/:id	Artwork details, bid form, history	Public
Login	/login	Authentication form	Guest
Register	/register	Registration form with role selection	Guest
Profile	/profile	Edit profile, view stats	Authenticated
Watchlist	/watchlist	Saved auctions	BUYER
Notifications	/notifications	Real-time alerts list	Authenticated
Create Auction	/create-auction	Upload artwork + create auction	ARTIST
Not Found	*	404 page	Public

4.2 Software Interfaces

Interface	Technology	Purpose
Frontend-Backend	REST API (HTTP/JSON)	CRUD operations

Interface	Technology	Purpose
Real-Time	WebSocket (STOMP/SockJS)	Bid updates, notifications
Service-to-Service	OpenFeign (HTTP)	Auction → Notification calls
Database	JDBC/PostgreSQL	Data persistence
Service Discovery	Eureka Client	Service registration

4.3 Communications Interfaces

- **Protocol:** HTTP/1.1 (REST), WebSocket
- **Data Format:** JSON (application/json)
- **Authentication:** Bearer Token (JWT) in Authorization header
- **CORS:** Configured for frontend origin
- **Proxy:** Vite dev server proxies /api to http://localhost:8080

5. Non-Functional Requirements

5.1 Performance Requirements

- 1: API response time shall be < 500ms for 95% of requests under normal load.
- 2: WebSocket message delivery shall be < 100ms.
- 3: Page load time shall be < 2s on broadband connections.
- 4: System shall support minimum 100 concurrent users.
- 5: Auction scheduler shall process status updates every 30 seconds.

5.2 Security Requirements

- 1: All passwords shall be hashed using BCrypt with strength 10.
- 2: JWT tokens shall expire after 24 hours.
- 3: API Gateway shall validate JWT signature before routing.
- 4: HTTPS shall be used in production (nginx reverse proxy).
- 5: SQL injection shall be prevented via JPA/Hibernate parameterized queries.
- 6: XSS shall be prevented via React's built-in escaping.
- 7: CORS shall restrict origins to known frontend domains.

5.3 Availability & Reliability

- 1:** System shall achieve 99.9% uptime during business hours.
- 2:** Docker Compose shall restart failed services automatically (restart: unless-stopped).
- 3:** Database data shall persist across container restarts (volumes).
- 4:** Eureka shall maintain service registry with heartbeat checks.

5.4 Maintainability

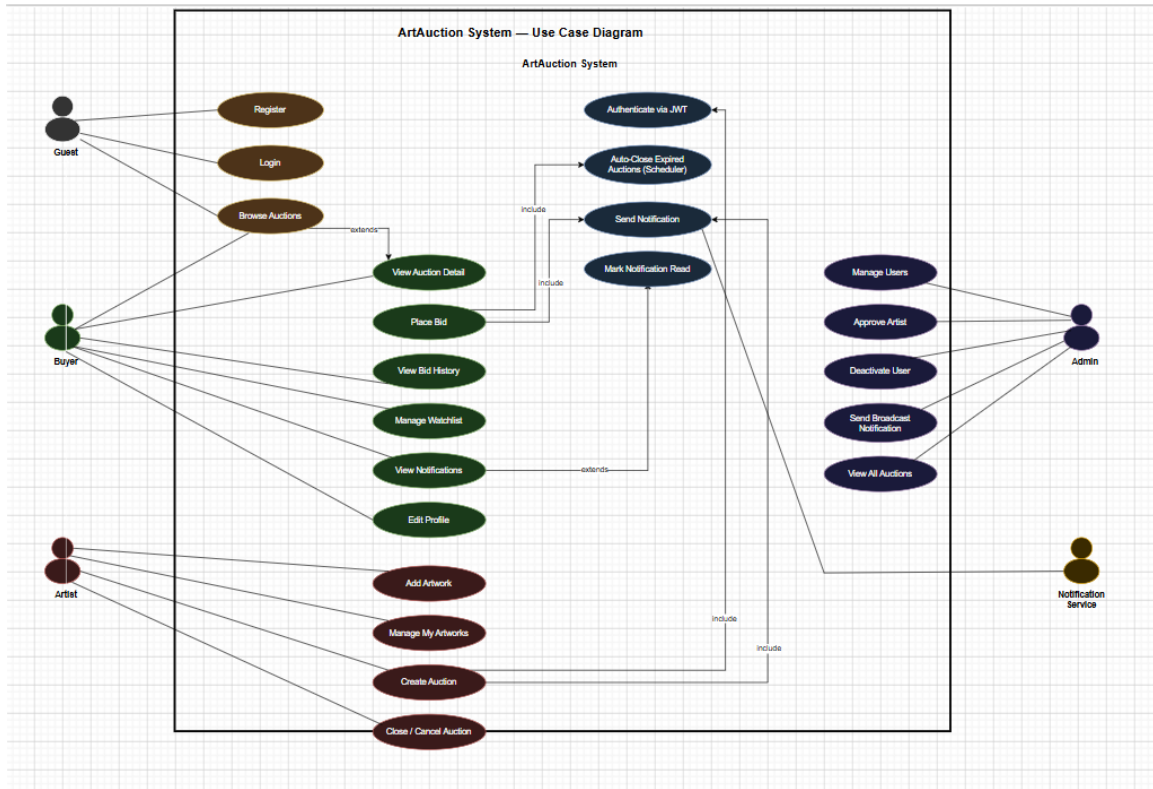
- 1:** Code shall follow Google Java Style Guide and Airbnb React Style Guide.
- 2:** Each microservice shall have independent CI/CD pipeline.
- 3:** API versioning shall be implemented (v1 prefix).
- 4:** Comprehensive logging shall be implemented via AOP.

5.5 Scalability

- 1:** Microservices shall be independently scalable via Docker Compose replica configuration.
- 2:** Stateless service design shall allow horizontal scaling.
- 3:** Database connection pooling shall be configured (HikariCP).

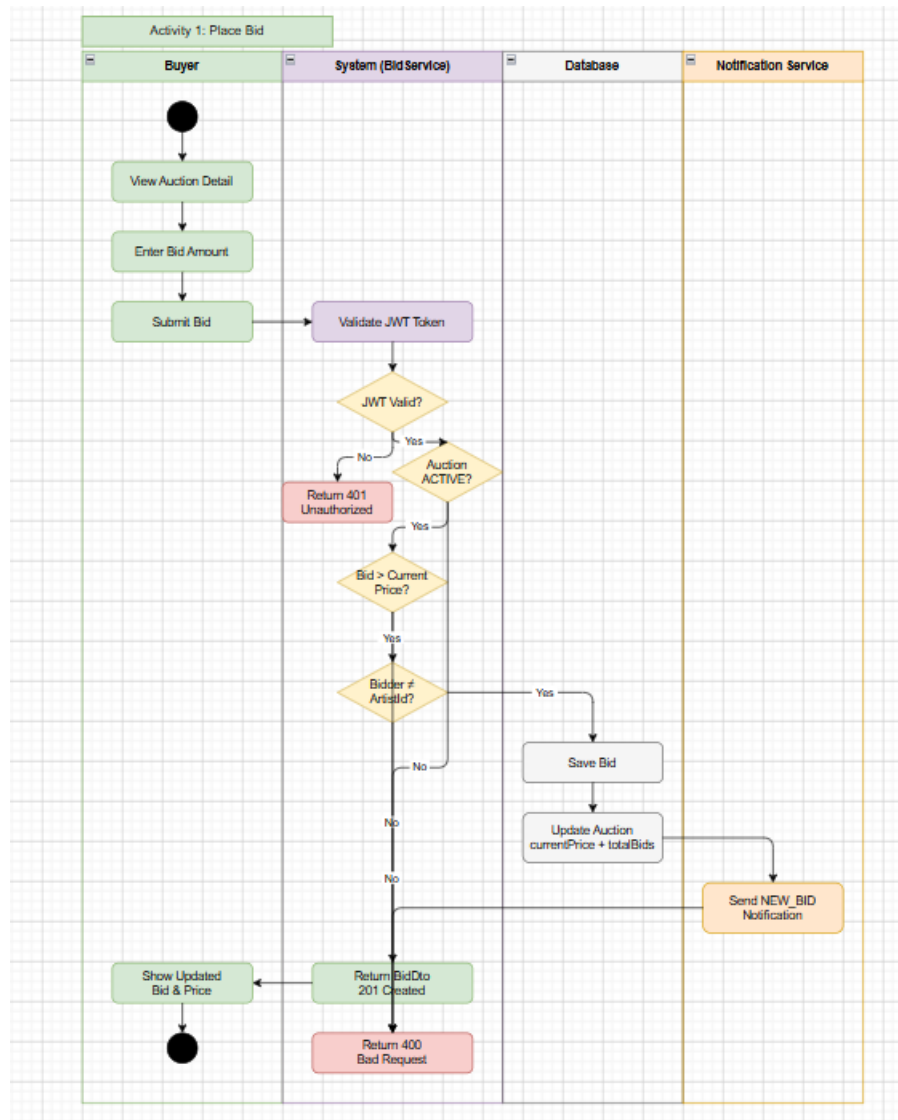
6. UML Diagrams

6.1 Use Case Diagram

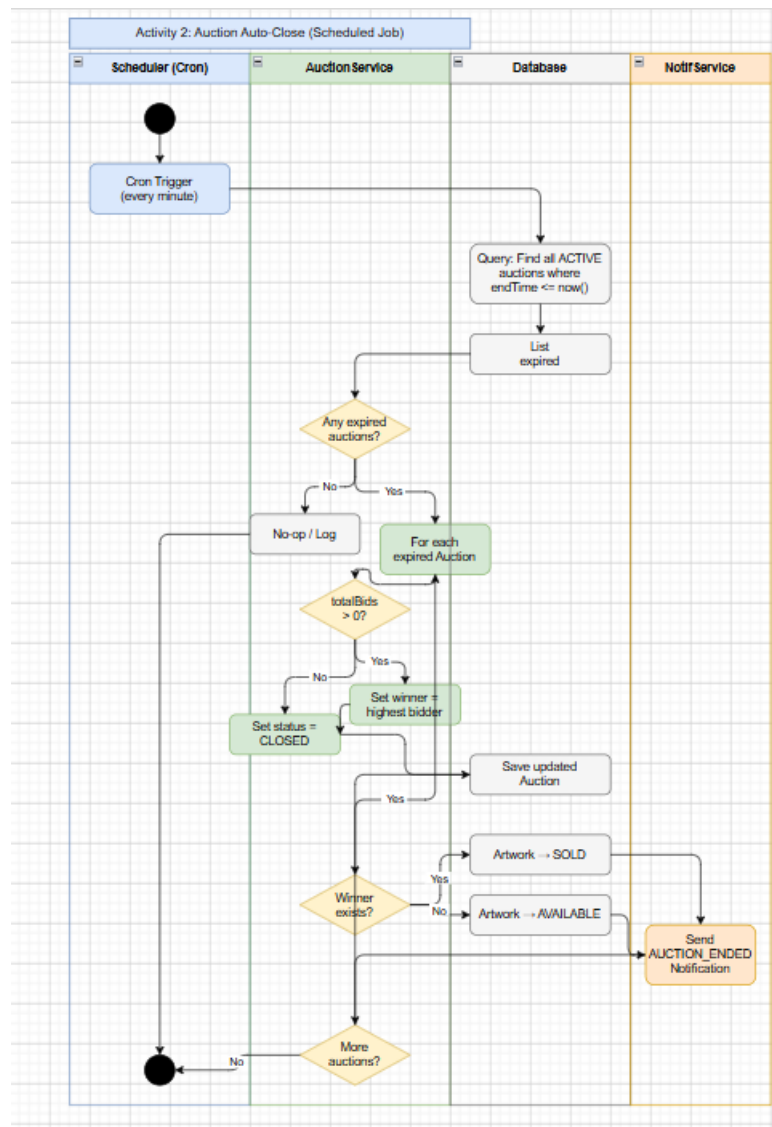


6.2 Activity Diagrams

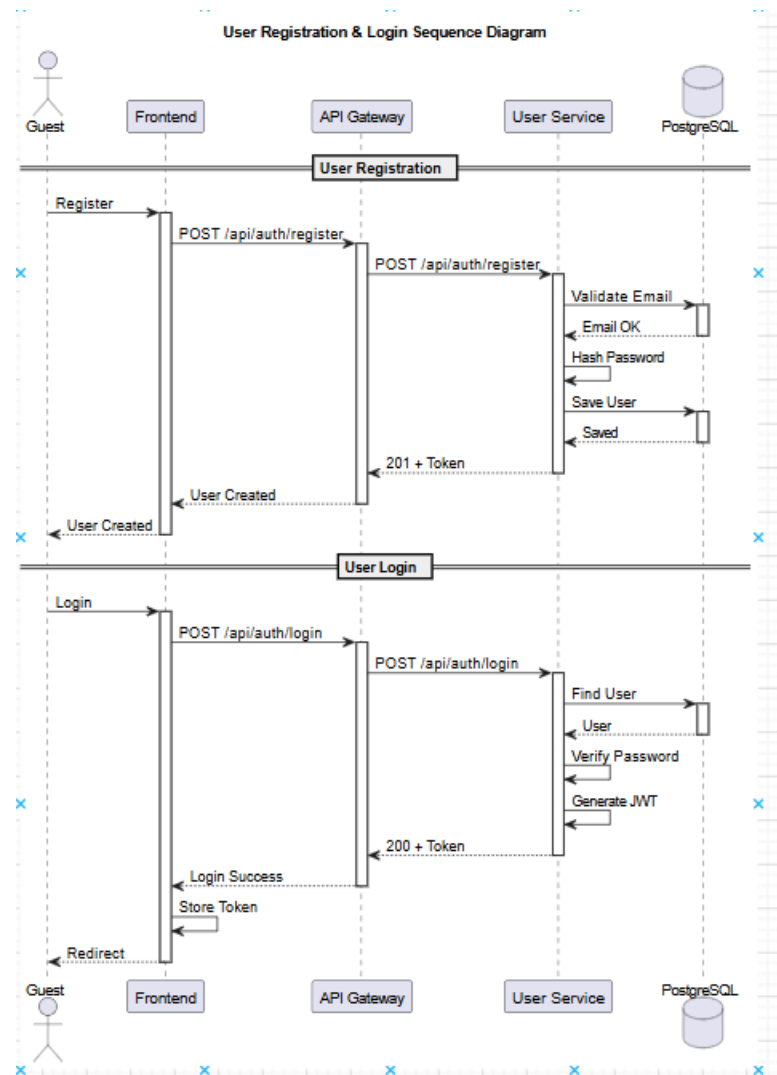
6.2.1 Place bid



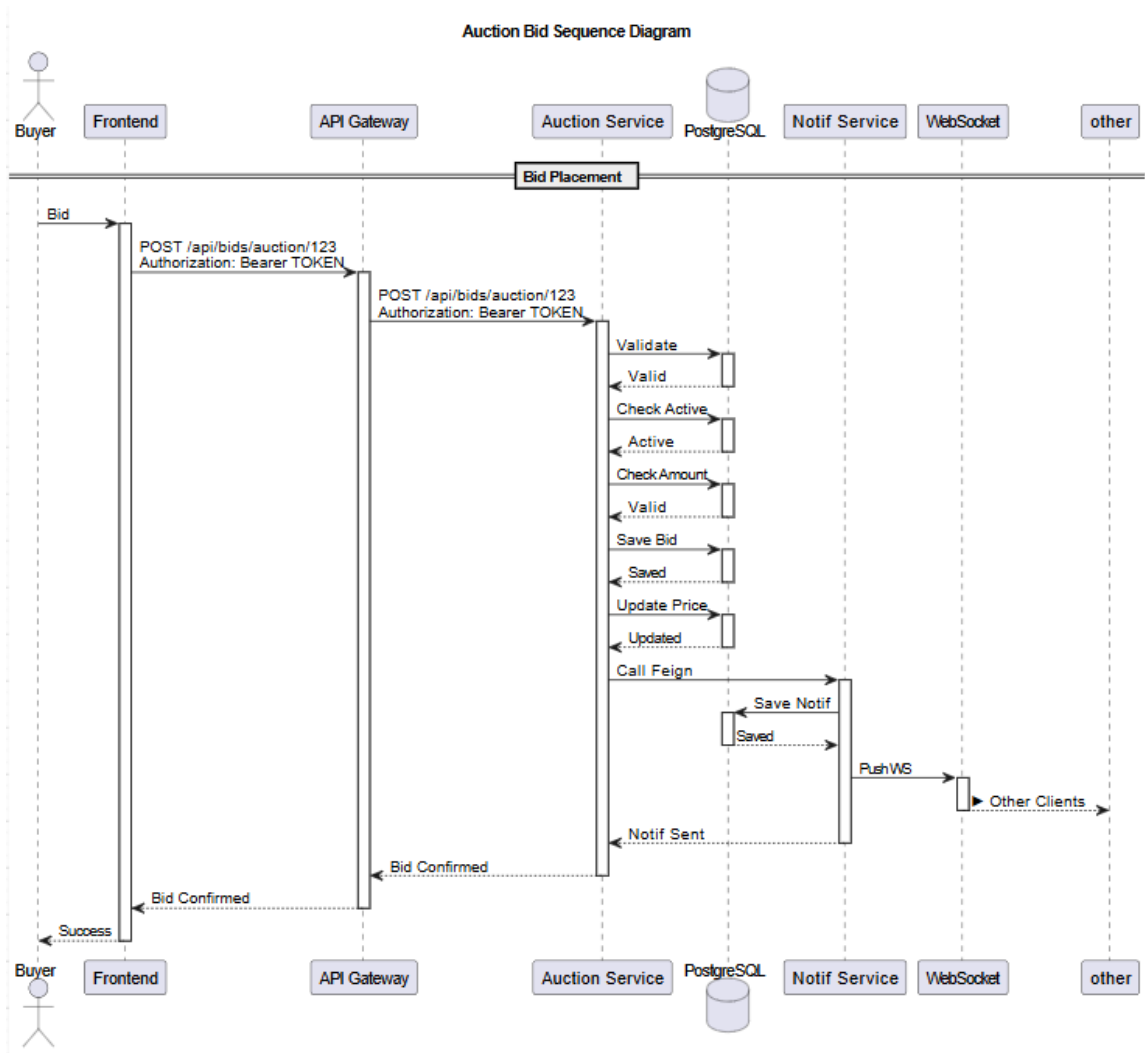
6.2.2 Auction Auto-close



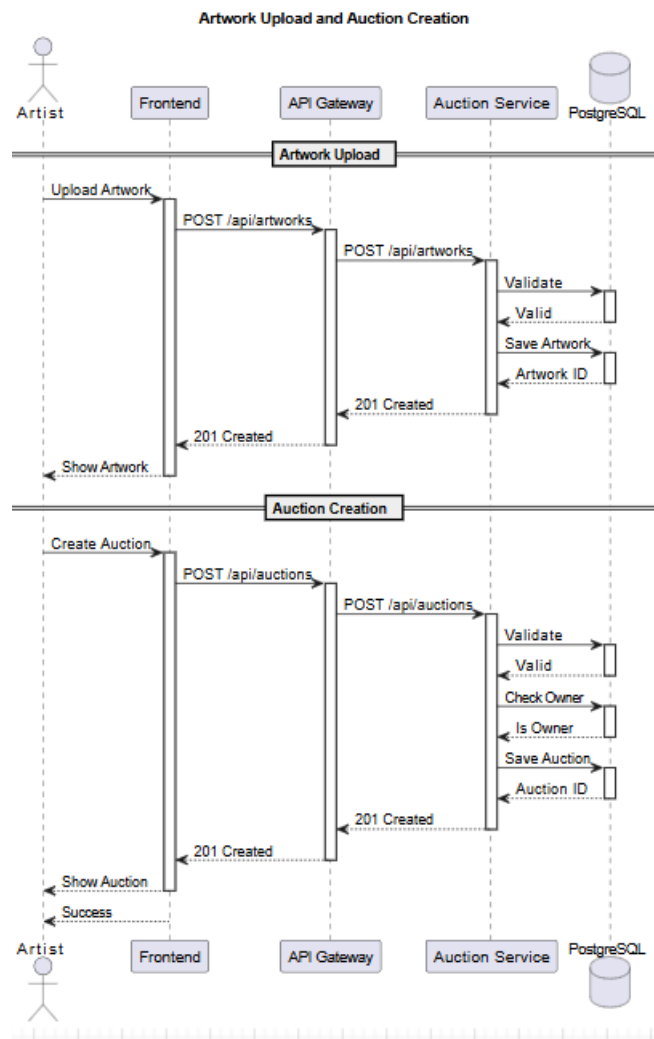
6.3 Sequence Diagrams



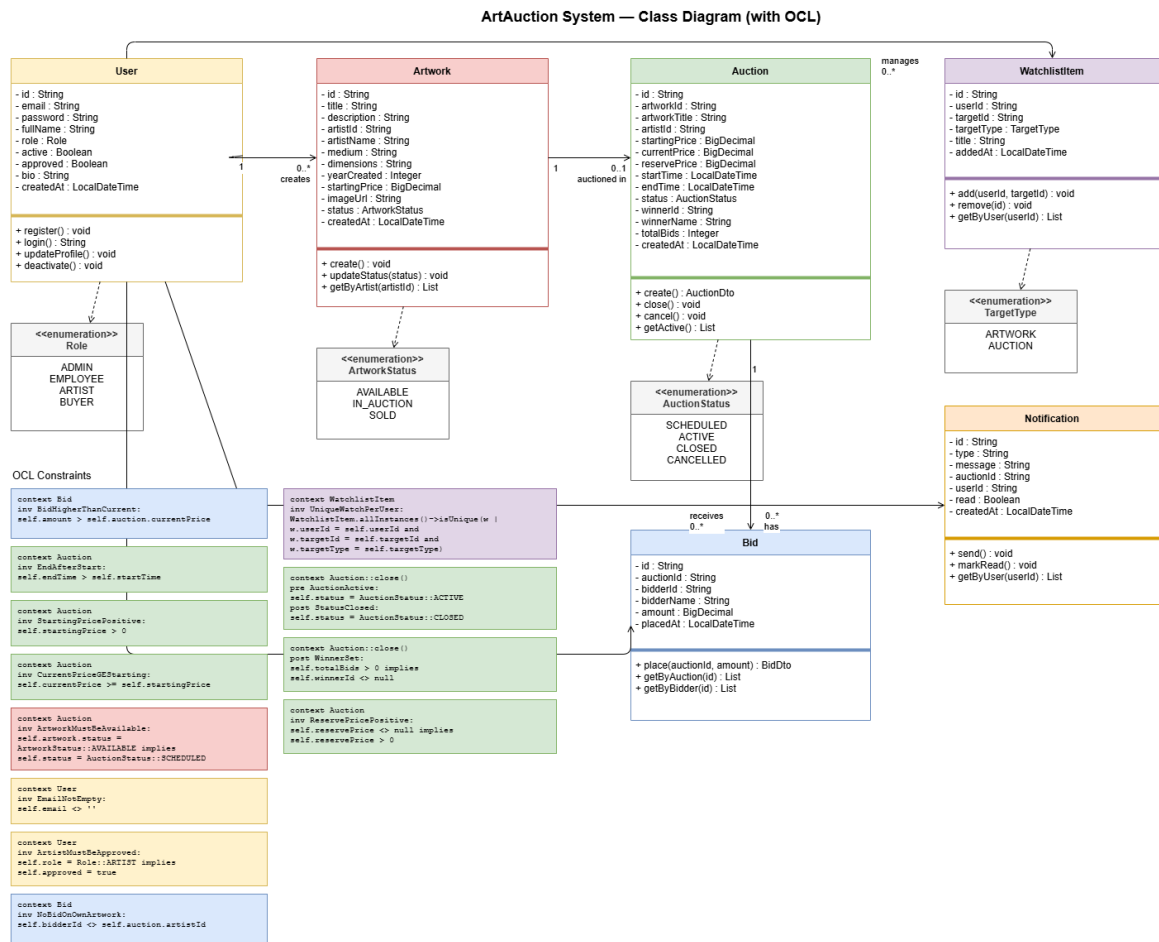
6.3.2 Place Bid & Real-Time Notification



6.3.3 Artist Upload Artwork & Create Auction

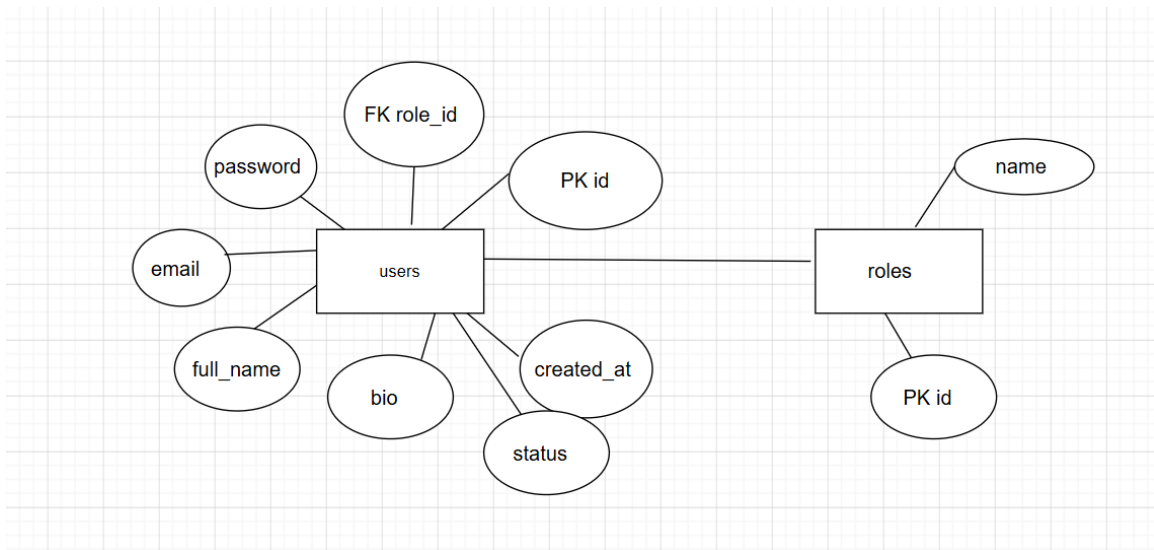


6.4 Class Diagram

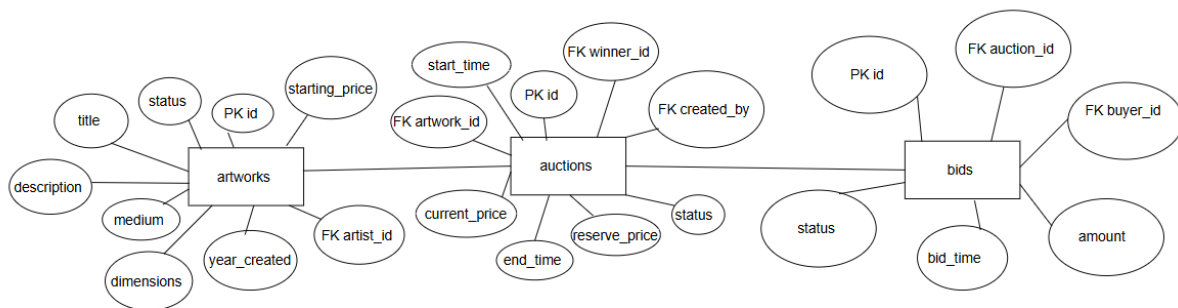


6.5 Entity-Relationship Diagram (ERD)

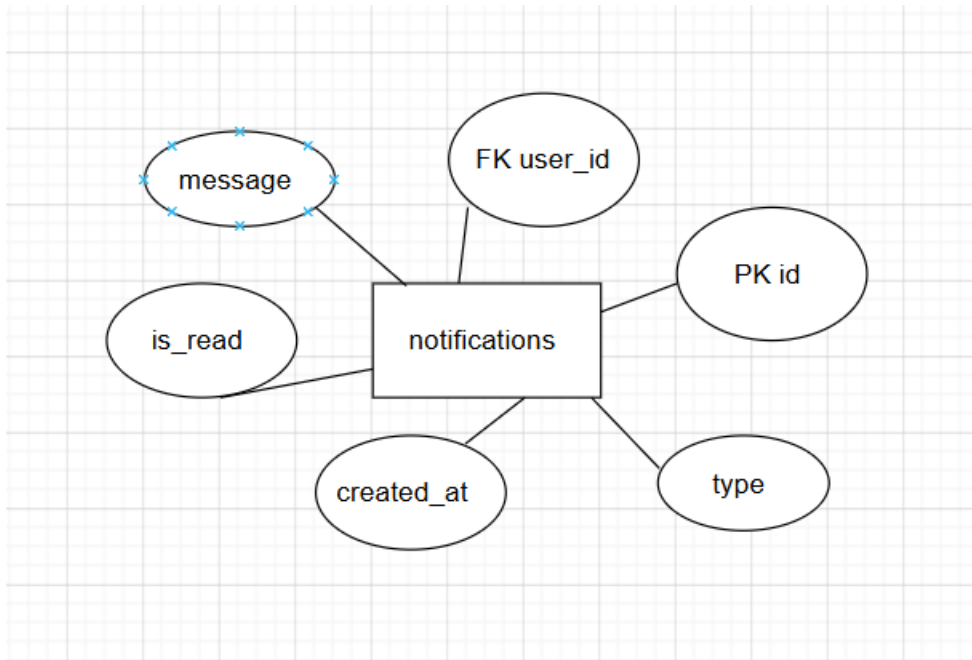
User service db



Auction service db



Notification service db



Relationships:

- users (1) — (M) artworks [Artist creates many artworks]
- artworks (1) — (1) auctions [One artwork per auction]
- auctions (1) — (M) bids [One auction has many bids]
- users (1) — (M) bids [One buyer places many bids]
- users (1) — (M) notifications [One user receives many notifications]

7. Implementation Details

7.1 API Specifications

7.1.1 Authentication Endpoints

Method	Endpoint	Auth	Request Body	Response	Description
POST	/api/auth/register	No	{fullName, email, password, role, bio?}	201 Created + User	Register new user
POST	/api/auth/login	No	{email, password}	200 OK + {token}	Authenticate user

7.1.2 User Management Endpoints

Method	Endpoint	Auth	Role	Description
GET	/api/users	Bearer	ADMIN/ EMPLOYEE	List all users
GET	/api/users/artists/pending	Bearer	ADMIN/EMPLOYEE	List pending artists
PUT	/api/users/artists/{id}/approve	Bearer	ADMIN/EMPLOYEE	Approve artist
DELETE	/api/users/{id}	Bearer	ADMIN	Delete user

7.1.3 Artwork Endpoints

Method	Endpoint	Auth	Role	Request Body	Description
GET	/api/artworks	No	Any	-	List all artworks
POST	/api/artworks	Bearer	ARTIST	Artwork JSON	Upload artwork
DELETE	/api/artworks/{id}	Bearer	EMPLOYEE/ADMIN	-	Delete artwork

7.1.4 Auction Endpoints

Method	Endpoint	Auth	Role	Request Body	Description
GET	/api/auctions	No	Any	-	List all auctions
GET	/api/auctions/active	No	Any	-	List active auctions
POST	/api/auctions	Bearer	ARTIST	Auction JSON	Create auction
PUT	/api/auctions/{id}/cancel	Bearer	ARTIST/EMPLOYEE	-	Cancel auction

Method	Endpoint	Auth	Role	Request Body	Description
			OYEE/ ADMIN		

7.1.5 Bid Endpoints

Method	Endpoint	Auth	Role	Request Body	Description
POST	/api/bids/auction/{id}	Bearer	BUYER	{amount}	Place bid
GET	/api/bids/auction/{id}	No	Any	-	Bid history
GET	/api/bids/my	Bearer	Any	-	My bid history

7.1.6 Notification Endpoints

Method	Endpoint	Auth	Role	Description
GET	/api/notifications/all	Bearer	ADMIN	All notifications
GET	/api/notifications/my	Bearer	Any	My notifications
PUT	/api/notifications/{id}/read	Bearer	Any	Mark as read

7.2 Object Constraint Language (OCL)

The following constraints define the business rules and validation logic for the ArtAuction system using Object Constraint Language principles expressed in natural language.

7.2.1 User Constraints

Invariant: Unique Email Address Every user in the system must have a unique email address. No two users can share the same email, ensuring proper identification and preventing duplicate accounts.

Invariant: Minimum Password Length All user passwords must contain at least 6 characters. This enforces a basic security standard for account protection.

Invariant: Artist Bio Requirement Any user who registers with the ARTIST role must provide a bio. The bio field cannot be left empty, as it helps buyers learn about the artist before purchasing their work.

Invariant: Valid Status Transitions A user account must always be in one of the following valid states: ACTIVE, PENDING, REJECTED, or SUSPENDED. Additionally, the PENDING status is only allowed for users with the ARTIST role who are awaiting approval from an administrator or employee.

Precondition for Login: Before a user can log in, their account status must be ACTIVE. Users with PENDING, REJECTED, or SUSPENDED status cannot access the system.

Postcondition for Login: After a successful login attempt, the system must verify that the provided password matches the hashed password stored in the database.

7.2.2 Artwork Constraints

Invariant: Positive Starting Price Every artwork must have a starting price greater than zero. Free artworks are not permitted on the platform.

Invariant: Valid Creation Year The year created for any artwork cannot be in the future. It must be the current year or any year in the past.

Invariant: Non-Empty Title Every artwork must have a title. The title field cannot be null or empty, as it is required for display and identification.

Precondition for Artwork Deletion: An artwork can only be deleted if it is not currently linked to an active auction. If an auction is ongoing for that artwork, deletion is blocked to maintain auction integrity.

Postcondition for Artwork Deletion: Once an artwork is successfully deleted, its status must be updated to DELETED in the system.

7.2.3 Auction Constraints

Invariant: Valid Time Range The end time of an auction must always be after the start time. An auction cannot end before or at the same moment it begins.

Invariant: Positive Reserve Price Every auction must have a reserve price greater than zero. The reserve price represents the minimum acceptable bid.

Invariant: Current Price Above Reserve (When Active) If an auction is currently ACTIVE, its current price must always be greater than or equal to the reserve price. This ensures that no winning bid falls below the seller's minimum acceptable amount.

Invariant: Winner Assignment on Close When an auction status changes to CLOSED, a winner must be assigned. The winner ID cannot remain null once the auction has concluded.

Preconditions for Placing a Bid: Before a bid can be placed, three conditions must be met: the auction must be in ACTIVE status, the bid amount must be greater than the current highest bid, and if this is the first bid, the amount must meet or exceed the reserve price.

Postconditions for Placing a Bid: After a valid bid is placed, the auction's current price must be updated to the new bid amount. The bid record must store the correct amount, the buyer's ID, and be marked with ACTIVE status.

7.2.4 Bid Constraints

Invariant: Positive Bid Amount Every bid amount must be greater than zero. Negative or zero bids are not allowed.

Invariant: Valid Bid Timestamp The timestamp recorded for a bid cannot be in the future. It must reflect the actual time when the bid was placed.

Precondition for Bid Cancellation: A bid can only be cancelled if it is currently in ACTIVE status. Bids that have already been processed or closed cannot be cancelled.

Postcondition for Bid Cancellation: After a bid is successfully cancelled, its status must be updated to CANCELLED.

7.2.5 Notification Constraints

Invariant: Non-Empty Message Every notification must contain a message. The message field cannot be null or empty, as it carries the information being communicated to the user.

Invariant: Valid Recipient User Every notification must be associated with an existing user. The system must verify that the user ID referenced in the notification corresponds to a valid user account in the database.

7.3 Aspect-Oriented Programming (AOP)

7.3.1 Implementation Overview

All four business services (User, Auction, Notification, Watchlist) implement cross-cutting concerns using Spring AOP. The logging aspect intercepts method execution without modifying business logic.

7.3.2 Logging Aspect (Common to All Services)

The Logging Aspect is implemented as a cross-cutting concern across all four business services: User Service, Auction Service, Notification Service, and Watchlist Service. It intercepts method calls without modifying the actual business logic code.

Controller Method Logging:

The aspect automatically captures every request that enters a REST controller. When a controller method is invoked, the aspect records the class name, method name, and the input arguments being passed. It starts a timer to measure execution duration. If the method completes successfully, the aspect logs the exit timestamp, total duration in milliseconds, and the returned result. If an exception occurs during execution, the aspect catches the error, logs the exception message along with the duration, and then re-throws the exception so the normal error handling flow continues uninterrupted.

Service Method Logging:

Similarly, the aspect intercepts all methods within the service layer. For service methods, it logs the entry point with class and method identifiers, tracks the execution time, and records successful completion with duration metrics. Service-level exceptions are also captured and logged with their error messages before being propagated up the call stack.

Key Behaviors:

- Entry logging captures the moment a method begins execution, including the method identity and any input parameters.
- Exit logging records successful completion, showing how long the method took to execute and what result was produced.
- Exception logging captures any errors that occur, preserving the error message and execution duration for debugging purposes.
- The aspect uses different log levels: INFO for controller entry and exit events, DEBUG for service layer monitoring, and ERROR for any exceptions.
- Execution timing is measured using system timestamps, providing precise performance metrics for every intercepted method call.

7.3.3 Cross-Cutting Concerns Addressed

Concern	Aspect	Target	Join Point
Logging	LoggingAspect	All Services & Controllers	@Around method execution
Performance Monitoring	PerformanceAspect	Service methods	@Around with timing
Exception Handling	ExceptionAspect	Controller methods	@AfterThrowing
Security Audit	AuditAspect	Authentication methods	@AfterReturning

7.3.4 Benefits of AOP in ArtAuction

1. **Separation of Concerns:** Business logic remains clean without logging code
2. **Consistency:** All services use identical logging format
3. **Maintainability:** Change logging behavior in one place
4. **Debugging:** Complete request/response traceability
5. **Performance:** Execution time measurement without code pollution

7.4 Docker Configuration

7.4.1 Docker Benefits

Benefit	Description
Consistency	Same environment across dev, test, and production
Isolation	Each service runs in its own container
Scalability	Scale individual services independently
Portability	Run anywhere Docker is installed
Rapid Deployment	<code>docker compose up --build</code> starts entire stack

7.5 Clean Code Practices

7.5.1 Naming Conventions

Layer	Convention	Example
Controllers	Resource + Controller	AuctionController
Services	Resource + Service/Impl	AuctionService, AuctionServiceImpl
Repositories	Resource + Repository	AuctionRepository
Entities	PascalCase, singular	Auction, Bid
DTOs	Resource + Dto/Request/Response	AuctionRequest, BidResponse
Methods	camelCase, verb-first	placeBid(), cancelAuction()
Constants	UPPER_SNAKE_CASE	DEFAULT_PAGE_SIZE

7.5.2 SOLID Principles Applied

Principle	Application
Single Responsibility	Each service handles one domain (User, Auction, Notification)

Principle	Application
Open/Closed	Extensible via interfaces (UserService interface, UserServiceImpl class)
Liskov Substitution	Repository interfaces allow swapping JPA implementations
Interface Segregation	Separate DTOs for Request/Response (not bloated objects)
Dependency Inversion	Services depend on abstractions (interfaces), not concrete classes

7.6 Design Patterns

7.6.1 Creational Patterns

Pattern	Location	Purpose
Singleton	JwtTokenProvider, EurekaServer	Single instance across application
Factory	UserFactory (registration)	Create users based on role type
Builder	Lombok @Builder on entities	Fluent object construction

7.6.2 Structural Patterns

Pattern	Location	Purpose
Repository	JpaRepository interfaces	Abstract data access layer
Facade	AuctionService	Simplify complex auction operations
Proxy	Spring @Transactional	Transaction management proxy
Adapter	DTO mappers (MapStruct)	Convert between entity and DTO

7.6.3 Behavioral Patterns

Pattern	Location	Purpose
Observer	WebSocket/STOMP	Real-time bid updates to clients
Strategy	JwtAuthFilter	Different auth strategies per endpoint
Template Method	LoggingAspect	Common logging template for all methods
Command	REST API endpoints	Encapsulate requests as objects

7.6.4 Microservices Patterns

Pattern	Implementation	Description
API Gateway	Spring Cloud Gateway	Single entry point, JWT validation
Service Discovery	Eureka Server	Dynamic service registration
Circuit Breaker	Resilience4j (planned)	Fault tolerance between services
Database per Service	4 PostgreSQL DBs	Data isolation per microservice
Event-Driven	WebSocket + Feign	Async notification delivery

7.7 Microservices & Cloud Architecture

7.7.1 Architecture Overview

The ArtAuction platform follows the **Microservices Architecture** pattern, decomposing the application into independently deployable services.

7.7.2 Service Responsibilities

Service	Port	Database	Responsibilities
Eureka Server	8761	-	Service registry, heartbeat monitoring
API Gateway	8080	-	JWT validation, routing, load balancing
User Service	8081	user_service_db	Auth, registration, user management, artist approval
Auction Service	8082	auction_service_db	Artworks, auctions, bids, scheduler
Notification Service	8083	notification_service_db	Notifications, WebSocket delivery
Watchlist Service	8084	watchlist_service_db	Watchlist CRUD, auction validation, bid notifications

7.7.3 Inter-Service Communication

From	To	Protocol	Purpose
API Gateway	All Services	HTTP (lb://)	Route requests via Eureka
Auction Service	Notification Service	OpenFeign (HTTP)	Send bid notifications
Watchlist Service	Auction Service	OpenFeign (HTTP)	Validate auction exists
Watchlist Service	Notification Service	OpenFeign (HTTP)	Notify watchers of new bids
Frontend	API Gateway	HTTP/WebSocket	User requests, real-time updates
All Services	Eureka Server	HTTP	Service registration/heartbeat



ArtAuction > Register Employee

POST http://localhost:8080/api/auth/register

Body

```
1 {
2   "fullName": "employee one",
3   "email": "employee@art.com",
4   "password": "staff123",
5   "role": "EMPLOYEE"
6 }
```

200 OK • 132 ms • 863 B

Body

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlbXBsY31lZUBhcnQyY29tIiwidXNlcikIjo1NDAA4yJ1MDQtZjE4YS00ODkzLWlzzTItMDdjYTlhMmUzZDBjIiwicm9sZSI6IkVNUExPWUVFIiw1ZnVsE5hbmU1Oj1lbXBsY31lZSBybmU1LCpYXQ1OjE3NzgyOTM5NjQsImV4cCI6MTc3ODM4MDM2NH9.TIGoRSSwV5jeiAts8eG9u1w8phCz1wm72aQhKUIQNg",
3   "userId": "488b5504-f18a-4893-b3e2-07ca3a1e3d0c",
4   "email": "employee@art.com",
5   "fullName": "employee one",
6   "role": "EMPLOYEE"
7 }
```

Get Pending Artists

ArtAuction > Get Pending Artists

GET http://localhost:8080/api/users/artists/pending

Authorization

Auth Type: Bearer Token

Token:

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

200 OK • 695 ms • 2.19 KB

Body

```
29 {
30   "bio": "",
31   "createdAt": "2026-05-08T18:10:32.149578"
32 },
33 {
34   "id": "240cfe7f-a42a-4d7d-8520-afd71849d96e",
35   "email": "ganna@gmail.com",
36   "fullName": "ganna ",
37   "role": "ARTIST",
38   "active": true,
39   "approved": false,
40   "bio": "",
41   "createdAt": "2026-05-08T18:24:37.347966"
42 }
```

Approve Artist

ArtAuction > Approve Artist

PUT

http://localhost:8080/api/users/artists/d9897094-8c1b-45c6-8c44-165282d5e7b9/approve

Send

DocsParamsAuthorizationHeaders9BodyScriptsTestsSettings

Auth Type

Bearer Token

Token

.....

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

BodyCookiesHeaders15Test Results

200 OK · 33 ms · 691 B

Save Response

JSONPreviewVisualize

```
1 {
2   "id": "d9897094-8c1b-45c6-8c44-165282d5e7b9",
3   "email": "yousefmoahmed@gmail.com",
4   "fullName": "youssef mohamed",
5   "role": "ARTIST",
6   "active": true,
7   "approved": true,
8   "bio": "Modern abstract painter",
9   "createdAt": "2026-05-09T02:47:01.204929"
10 }
```

Login as ARTIST

The screenshot shows the ArtAuction API client interface. At the top, there's a header bar with "ArtAuction > Login ARTIST". Below it, a dropdown menu is set to "POST" and the URL is "http://localhost:8080/api/auth/login". A "Send" button is visible. On the left, navigation tabs include Docs, Params, Authorization, Headers, Body (selected), Scripts, Tests, and Settings. Below these are radio buttons for request types: none, form-data, x-www-form-urlencoded, raw (selected), binary, GraphQL, and JSON. The main area displays the raw request body as a JSON object:

```
{
  "email": "yousefmohamed@gmail.com",
  "password": "youssef123"
}
```

At the bottom, the response section shows a status of "200 OK" with headers like "Content-Type: application/json" and "Access-Control-Allow-Origin: *". The response body is a JSON object:

```
{
  "token": "eyJzdmFIOTIjbnVzZWZtb2hhbWVkQGdtYWIzLnMvbmSI6InVzZXJ3ZCI6ImQ5ODk3MDk0LThjMWItNDVjNjE0Y2Y0QlRlTE2NTI4MmQ1ZTdlOStIsInjBdGU1OjBULURJU0lCJmdWxsTmFtZSI6InlvXGZnYyYgby9oYWllZCI6ImhhdCI6MTc3ODU5NDkzNSwiZXhwIjoxejoxNzc4MzgxmZm1fOi5rKP19MDIzLTZLOdbvF2z9Ha_0-UT4I-XS865dZCZko",
  "userId": "d9897094-8c1b-45c6-8c4d-165282d5e7b9",
  "email": "yousefmohamed@gmail.com",
  "fullName": "youssef mohamed",
  "role": "ARTIST"
}
```

Upload Artwork

The screenshot shows a REST client interface for a service named 'ArtAuction'. The request is a POST to 'http://localhost:8080/api/artworks'. The request body is a JSON object with the following fields: 'title' (Sunrise Over Nile), 'description' (Oil painting capturing morning light), 'medium' (Oil on Canvas), 'dimensions' (80x60cm), 'yearCreated' (2026), and 'startingPrice' (100.00). The response is a 200 OK status with a response time of 155 ms and a body size of 842 B. The response body is a JSON object with the following fields: 'id' (d98871d1-38a1-427a-8895-5d2af4c0cdea), 'title' (Sunrise Over Nile), 'description' (Oil painting capturing morning light), 'artistId' (d9897094-8c1b-45c6-8c44-165282d5e7b9), 'artistName' (youssef mohamed), 'medium' (Oil on Canvas), 'dimensions' (80x60cm), 'yearCreated' (2026), 'startingPrice' (100.00), 'imageUrl' (null), 'status' (AVAILABLE), and 'createdAt' (2026-05-09T02:54:21.720262217).

```
POST http://localhost:8080/api/artworks

{
  "title": "Sunrise Over Nile",
  "description": "Oil painting capturing morning light",
  "medium": "Oil on Canvas",
  "dimensions": "80x60cm",
  "yearCreated": 2026,
  "startingPrice": 100.00
}
```

```
{
  "id": "d98871d1-38a1-427a-8895-5d2af4c0cdea",
  "title": "Sunrise Over Nile",
  "description": "Oil painting capturing morning light",
  "artistId": "d9897094-8c1b-45c6-8c44-165282d5e7b9",
  "artistName": "youssef mohamed",
  "medium": "Oil on Canvas",
  "dimensions": "80x60cm",
  "yearCreated": 2026,
  "startingPrice": 100.00,
  "imageUrl": null,
  "status": "AVAILABLE",
  "createdAt": "2026-05-09T02:54:21.720262217"
}
```

Create Auction

The screenshot shows a REST client interface for a service named 'ArtAuction'. The request is a POST to 'http://localhost:8080/api/auctions'. The request has a Bearer Token authorization header. The response is a 200 OK status with a response time of 461 ms and a body size of 888 B. The response body is a JSON object with the following fields: 'id' (8017e3f6-99ab-42c3-be18-61bb92279649), 'artworkId' (d98871d1-38a1-427a-8895-5d2af4c0cdea), 'artworkTitle' (Sunrise Over Nile), 'artistId' (d9897094-8c1b-45c6-8c44-165282d5e7b9), 'startingPrice' (100.00), 'currentPrice' (100.00), 'reservePrice' (400.00), 'startTime' (2024-01-01T00:00:00), 'endTime' (2099-12-31T23:59:59), 'status' (ACTIVE), 'winnerId' (null), 'winnerName' (null), and 'totalBids' (0).

```
POST http://localhost:8080/api/auctions

Authorization: Bearer Token
```

```
{
  "id": "8017e3f6-99ab-42c3-be18-61bb92279649",
  "artworkId": "d98871d1-38a1-427a-8895-5d2af4c0cdea",
  "artworkTitle": "Sunrise Over Nile",
  "artistId": "d9897094-8c1b-45c6-8c44-165282d5e7b9",
  "startingPrice": 100.00,
  "currentPrice": 100.00,
  "reservePrice": 400.00,
  "startTime": "2024-01-01T00:00:00",
  "endTime": "2099-12-31T23:59:59",
  "status": "ACTIVE",
  "winnerId": null,
  "winnerName": null,
  "totalBids": 0
}
```

Login BUYER



ArtAuction > List Artworks

GET

http://localhost:8080/api/artworks

Send

Docs

Params

Authorization

Headers 7

Body

Scripts

Tests

Settings

Cookies

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 15

Test Results

200 OK • 46 ms • 1.19 KB

Save Response

JSON

Preview

Visualize

```
10  "yearCreated": 2024,
11  "startingPrice": 500.00,
12  "imageUrl": null,
13  "status": "IN_AUCTION",
14  "createdAt": "2026-05-06T23:27:46.697106"
15  },
16  {
17    "id": "d88871d1-38a1-427a-8895-5d2af4c8cdea",
18    "title": "Sunrise Over Nile",
19    "description": "Oil painting capturing morning light",
20    "artistId": "d9897894-8c1b-45c6-8c44-165282d5e7b9",
21    "artistName": "youssef mohamed",
22    "medium": "Oil on Canvas",
23    "dimensions": "80x60cm",
```

8.2 FRONTEND

ADMIN APPROVAL

Art Auction

Auctions

Watchlist

Notifications

Admin

Admin User

Admin Dashboard

Manage users and system settings

1

PENDING APPROVAL

8

ACTIVE USERS

9

TOTAL REGISTERED

Pending Approval

All Users

NAME	EMAIL	ROLE	REGISTERED	ACTION
yousef se	ym10@gmail.com	Bidder	5/9/2026	Approve

BID HISTORY AND PLACE A BID

[Back to Auctions](#)



ACTIVE

[Watch](#)

SUN

by Bodyyasser

Oil

Started May 9, 2026, 07:53 PM

Ends May 11, 2026, 07:53 PM

CURRENT BID

\$600.00

STARTING PRICE

\$500.00

TIME REMAINING

1d 20h

Description

Sun is beautiful

Place a Bid

Minimum bid: \$600.01

\$

Enter amount (min 600.01)

Place Bid

Bid History

#1

yousef mohamed hassan abd

May 9, 2026, 07:56 PM

\$600.00

#2

Kenzey walled

May 9, 2026, 07:54 PM

\$500.01

NOTIFICATIONS

Art Auction

Auctions

Watchlist

Notifications12

+ Sell Art

Bodyyasser

Notifications

Mark all as read (12)

You successfully logged in to ArtAuction.

May 9, 2026, 08:32 PM

New bid of \$600 by yousef mohamed hassan abd on SUN

May 9, 2026, 07:56 PM

You successfully logged in to ArtAuction.

May 9, 2026, 07:55 PM

New bid of \$500.01 by Kenzey walled on SUN

May 9, 2026, 07:54 PM

Your auction for "SUN" is now live!

May 9, 2026, 07:53 PM

You successfully logged in to ArtAuction.

May 9, 2026, 07:52 PM

SELL ART

 Art Auction

Auctions

Watchlist

Notifications12

+ Sell Art

Bodyyasser

Create New Auction

Select an artwork and set auction parameters

Select Artwork

You don't have any artworks yet.

Create Artwork First

Auction Details

Start Time *

mm/dd/yyyy --:-- --

End Time *

mm/dd/yyyy --:-- --

Reserve Price (\$)

Auction must run for at least 1 hour

WATCH LIST

 Art Auction

Auctions

Watchlist

Notifications3

Kenzey walled

My Watchlist

Track auctions and artworks you're interested in

ACTIVE

No Image

SUN

by Unknown

CURRENT BID

\$600.00

ENDS IN

1d 20h

Art

2 bids

View Auction →

SCHEDULED

No Image

fdfddf

by Unknown

CURRENT BID

\$3,223.00

ENDS IN

20d 19h

Art

0 bids

View Auction →

8.3 DATABASES

USERS

	id [PK] character varying (255)	active boolean	approved boolean	bio character varying (255)	created_at timestamp without time zone (6)	email character varying (
	1a9da706-6d78-4ce9-b567-d1eaba64...	true	true	cat	2026-05-09 20:25:00.168622	shakshoka6@gmail
2	3d03ddd1-a573-482a-8f72-6df4dbf2d2...	true	true		2026-05-09 20:30:43.732832	ym10@gmail.com
3	4f733f77-e2d4-4808-a277-db1380de2e...	true	true	[null]	2026-05-09 19:05:59.541467	admin@test.com
4	668ae833-5339-43f4-a8af-07fd26fb94e2	true	true		2026-05-09 18:56:28.48793	kenzeywalled@gm
5	6b156e43-1809-4d65-a5b0-c52267981...	false	true		2026-05-09 19:17:31.846782	saramooo2010@g
6	71db620f-b270-4d2b-8d94-39c2f188a0...	true	true		2026-05-09 19:36:25.357712	yabdelrahman729
7	935e94c0-23eb-4627-939f-0732c5e84...	true	true		2026-05-09 18:44:58.806856	1020081@stemeg
8	93f06978-0840-4f6e-82d4-5ece0cede4...	true	true		2026-05-09 19:01:14.290958	ym202723@gmail.

	created_at timestamp without time zone (6)	email character varying (255)	full_name character varying (255)	password character varying (255)	role character varying (255)
1	2026-05-09 20:25:00.168622	shakshoka6@gmail.com	shakshoka	\$2a\$10\$eQ3HkLW2vw.Wj6S6rgMinOeTu1FR52gmJ2QK20ZvQbTPC1EQb...	ARTIST
2	2026-05-09 20:30:43.732832	ym10@gmail.com	yousef se	\$2a\$10\$732VtGcX9h.6M36VG4K6KuG3RHb05z0M.tCP5Xp6UfkhNmB....	BUYER
3	2026-05-09 19:05:59.541467	admin@test.com	Admin User	\$2a\$10\$KX6tuI3KUT/5m.S/P4QVKLV8MHebgkAVODuILJ0UbCZAaUAsrjly	ADMIN
4	2026-05-09 18:56:28.48793	kenzeywalled@gmail.com	Kenzey walled	\$2a\$10\$SrE8Hex0WsYsKPr5xGRrweFv1418ErYCSuNQ2zRpI308cU.XehH...	BUYER
5	2026-05-09 19:17:31.846782	saramooo2010@gmail.com	Sara Mohamed	\$2a\$10\$WGFhuBR62PcIIT3IGGGonOY9Szfr07HCN0T8WTPyXhH06rZJA...	ARTIST
6	2026-05-09 19:36:25.357712	yabdelrahman729@gmail.c...	Bodyyasser	\$2a\$10\$K5qN8FWLo3wEcDs0eAhag.SowxASrrzVZs4xdOwZ4ZvzJ5j0p...	ARTIST
7	2026-05-09 18:44:58.806856	1020081@stemegypt.edu.eg	yousef mo	\$2a\$10\$JlnVhEg0momZQgprfgYaoedqYVgBINRSS05uN2IMPGsQIQYCa...	BUYER
8	2026-05-09 19:01:14.290958	ym202723@gmail.com	yousef mohamed hassan a...	\$2a\$10\$us//xzehTMpWcl.3pZHgdeMfdFSrmp4bQa1W9uYuETBz4Jf5CvC.	BUYER

NOTIFICATION

Data Output

Messages

Notifications

Showing rows: 1 to 51

Page No: 1

of 1

	id [PK] character varying (255)	action_id character varying (255)	created_at timestamp without time zone (6)	message character varying (1000)	read boolean
44	d2840aeb-ae79-4047-bcbf-77c5847e6...	[null]	2026-05-09 19:52:41.870189	You successfully logged in to ArtAuction.	false
45	d769f571-1f83-479b-8f9c-4423a8d15...	[null]	2026-05-09 19:32:29.699192	You successfully logged in to ArtAuction.	true
46	d78d9c73-1b53-49e8-9653-12922c3e...	[null]	2026-05-09 20:32:04.063087	You successfully logged in to ArtAuction.	false
47	dbf2f7e5-107d-47be-b689-6b71478b...	[null]	2026-05-09 20:31:51.466697	Your ArtAuction account has been approved! You can now log in.	false
48	e799311c-134b-4e92-b153-e655ad2b...	[null]	2026-05-09 20:34:36.139309	You successfully logged in to ArtAuction.	false
49	e9b3963c-3606-4518-882e-e2e966f48...	[null]	2026-05-09 18:48:35.102591	Welcome to ArtAuction, yousef mohamed hassan! Your account is pending admin approval.	false
50	f107e574-c2ff-435f-8953-567fcf842d5f	[null]	2026-05-09 20:25:25.321354	You successfully logged in to ArtAuction.	true
51	fe506eec-54f2-4e04-b0f4-a63574e411...	[null]	2026-05-09 19:12:01.921686	You successfully logged in to ArtAuction.	true

ARTWORKS

Data Output

Messages

Notifications

≡

+

Showing rows: 1 to 2Page No:

1

 of 1

	id [PK] character varying (255)	artist_id character varying (255)	artist_name character varying (255)	created_at timestamp without time zone (6)	description character varying (255)
1	494d6fa9-f2f7-4d81-b099-3514c54bae...	6b156e43-1809-4d65-a5b0-c52267981...	Sara Mohamed	2026-05-09 19:19:17.05434	
2	cea51e87-cfae-4196-b36f-90cbb91659...	71db620f-b270-4d2b-8d94-39c2f188a0...	Bodyyasser	2026-05-09 19:53:22.559137	Sun is bea

Total rows: 2

Query complete 00:00:00.195

CRLF

Ln 1, Col 1

AUCTIONS

